Arzel Andrei Israel
Lynxie Jagonob
John Walter Pediangco
John Christian Setubal
Emmanuel Siastres

Lab 5: Update, Delete

Parameters

- the endpoint was a fixed path (like /api/products). But to update a specific product, the client needs to tell the server *which* one. We use a colon (:) in Express to create a variable in the URL, like /api/products/:id.

Open your server.js and add the following two logical blocks.

1. Update Endpoint (PUT)
    a. **The Route:** Define a route using app.put('/api/products/:id', ...).
    b. **Capture the ID:** Inside the function, access the ID from the URL using req.params.id.
    c. **Capture the Data:** Get the new name and price from the req.body (the payload).
    d. **The SQL Logic:** Prepare a SQL UPDATE statement.
        i. *Hint:* Your SQL should look something like: UPDATE products SET name = ?, price = ? WHERE id = ?.
    e. **The Check:** Use the .run() method. This returns an object. Check the .changes property of that object.
        i. *Logic:* If changes are 0, it means the ID didn't exist in the database. Send a 404 Not Found status. Otherwise, send a 200 OK.

```
// UPDATE
app.put('/api/products/:id', (req, res) => {
    const id = req.params.id;
    const { name, price } = req.body;

    const result = db.prepare(
        'UPDATE products SET name = ?, price = ? WHERE id = ?'
    ).run(name, price, id);

    if (result.changes === 0) {
        return res.status(404).json({ message: "Not found" });
    }

    res.json({ message: "Updated successfully" });
});
```

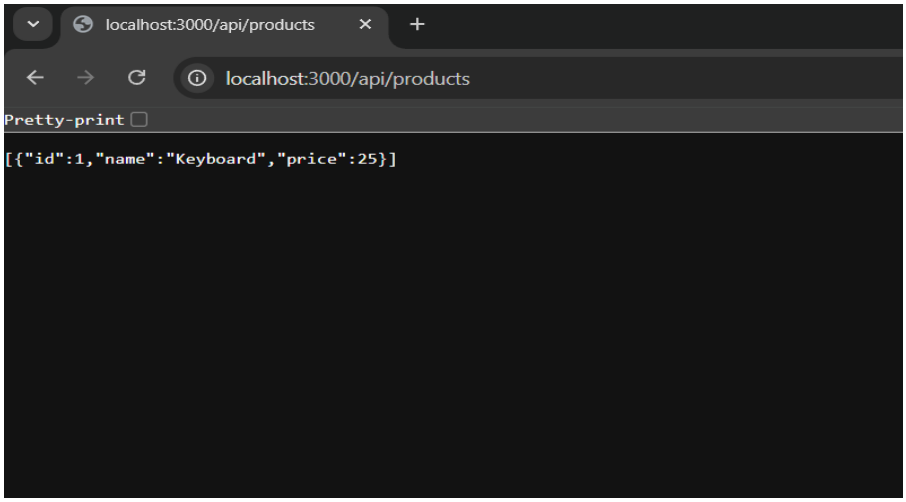
2. Delete Endpoint
    a. **The Route:** Define a route using app.app.delete('/api/products/:id', ...).
    b. **The SQL Logic:** Prepare a SQL DELETE statement targeting the id.
        i. *Hint:* DELETE FROM products WHERE id = ?.
    c. **The Execution:** Just like the PUT method, check if any rows were actually deleted.
        i. *Logic:* If the product was successfully removed, send a confirmation message. If the ID wasn't found, send a 404.

```
// DELETE
app.delete('/api/products/:id', (req, res) => {
    const id = req.params.id;

    const result = db.prepare(
        'DELETE FROM products WHERE id = ?'
    ).run(id);

    if (result.changes === 0) {
        return res.status(404).json({ message: "Not found" });
    }

    res.json({ message: "Deleted successfully" });
});
```

3. Testing with curl
    a. **Start your server:** node server.js.

```
PS C:\Users\emman\OneDrive\Documents\Setubal_BSIT3B_lab_5\lab 5> node server.js
Server running at http://localhost:3000
```

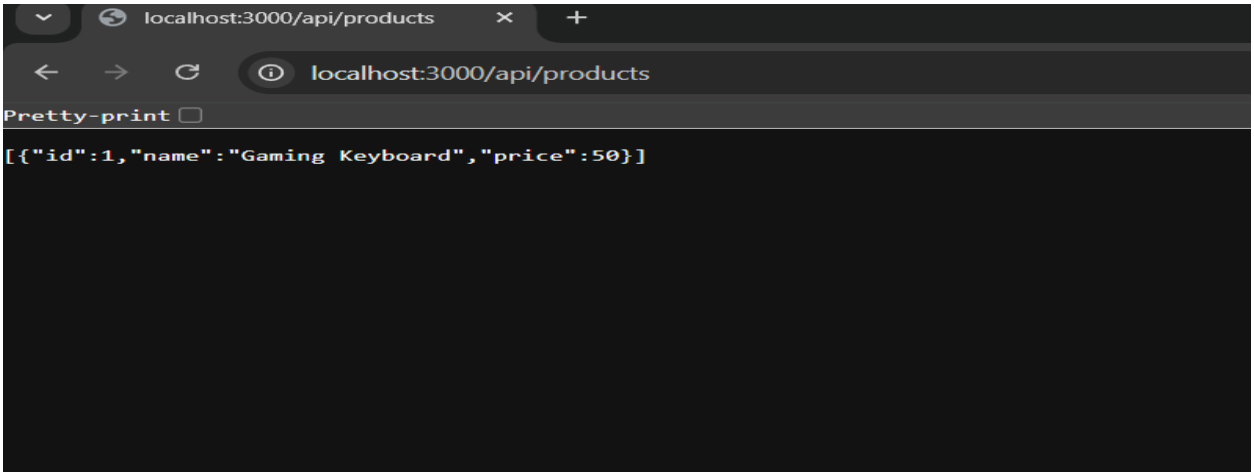
    b. **Verify your current data:** Go to http://localhost:3000/api/products to see which IDs are currently in your database.



c. **Perform an Update**

i. Change Product #1 to a "Gaming Keyboard" priced at "50".

```
PS C:\Users\emman\OneDrive\Documents\Setubal_BSIT3B_lab_5\lab 5> irm -Method PUT http://localhost:3000/api/products/1 `
>> -Headers @{ "Content-Type" = "application/json" } `
>> -Body '{"name":"Gaming Keyboard","price":50}'

message
-------
Updated successfully


PS C:\Users\emman\OneDrive\Documents\Setubal_BSIT3B_lab_5\lab 5>
```

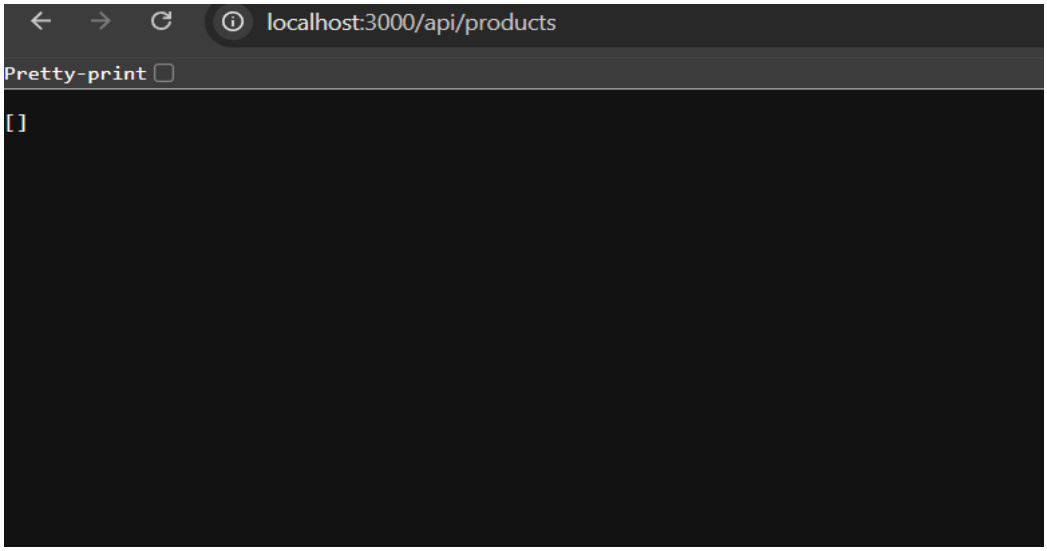


d. **Perform a Deletion**

i. Delete a different product

```
PS C:\Users\emman\OneDrive\Documents\Setubal_BSIT3B_lab_5\lab 5> irm -Method DELETE http://localhost:3000/api/products/1

message
-------
Deleted successfully


PS C:\Users\emman\OneDrive\Documents\Setubal_BSIT3B_lab_5\lab 5>
```

e. **Final Check:** Refresh your browser.



4. Questions
    a. Why is PUT used for updating data instead of POST? Think about the "Strict Synchronous Contract".

       - PUT is used to update existing data, while POST is used to create new data.

    b. In your DELETE logic, why is it better to return a 404 error if the ID doesn't exist, rather than just sending a "Success" message anyway?

       - Returning a 404 accurately communicates with the database's actual state. Sending a fake "Success" message masks potential bugs and makes troubleshooting harder.

    c. You've now built a full CRUD API. If you were to connect a React or Mobile app to this, which part of the "Modern API Architecture" would this Node.js server represent?

       - This Node.js server represents the Backend.